

Attack-Type-Specific Evaluation of Neural and Traditional Models for Flow-Level Intrusion Detection

Yonghyeon Lee
yhylee@ucdavis.edu
University of California, Davis
Davis, California, USA

Abstract

Network intrusion detection studies often report aggregate model performance, but a security operator needs to know which attacks are still missed. This paper evaluates whether neural models change the false-negative profile of flow-level intrusion detection compared with traditional machine learning baselines. Using the CSE-CIC-IDS2018 processed flow-feature dataset, I train Logistic Regression, Random Forest, Shallow MLP, Lightweight PyTorch MLP, Deep MLP, and Autoencoder + MLP models on a binary benign/malicious task, while preserving original attack labels for post-hoc analysis. The evaluation compares precision, recall, F1-score, false-negative rate, confusion matrices, fine-grained attack-type detection rates, and coarse attack-family detection rates. The main result is not that neural models are uniformly better. Deep MLP and Autoencoder + MLP improve precision and F1-score, but Logistic Regression has the highest attack recall and lowest false-negative rate. Across attack categories, DoS/DDoS and many brute-force attacks are detected well, while Infiltration remains a major blind spot for every model.

Keywords

intrusion detection, neural networks, Random Forest, false negatives, CSE-CIC-IDS2018, attack-type evaluation

1 Introduction

Network intrusion detection systems classify network activity as benign or suspicious so that defenders can investigate or block malicious traffic. In many machine learning studies of intrusion detection, the headline result is an aggregate score such as accuracy or F1-score. That style of reporting is useful for comparing models at a high level, but it is incomplete for security analysis. In an operational IDS, different mistakes have different costs. A false positive creates analyst workload, but a false negative means malicious traffic passed through without being flagged. Even if a model has a strong aggregate score, it may still systematically miss an attack family that is operationally important.

This project studies that problem in the setting of flow-level intrusion detection. Flow-level datasets summarize network traffic using features such as packet counts, byte counts, flow duration, rates, inter-arrival timing, and TCP flag behavior. These features are much easier to use in a quarter-scale reproducible study than raw packet traces, and they support both traditional machine learning and neural network models. At the same time, flow-level data has a risk: aggregate binary classification can hide large differences across attack categories.

The research question is: *which types of network intrusions are detected better by neural models compared with traditional machine*

learning baseline models on flow-level network traffic data? More specifically, the project asks whether model choice changes detection performance across attack families such as DoS/DDoS, brute force, botnet, infiltration, and web-based attacks. This is not a simple question of whether a neural model wins overall. Instead, the goal is to understand the security-relevant failure pattern of each model.

I hypothesized that neural models would improve detection for attack categories whose signatures depend on nonlinear combinations of flow-level features. For example, botnet or web-based attacks may not be captured by a single threshold over one feature. However, I also expected traditional models, especially Random Forest, to remain competitive on attacks with clearer separable feature patterns, such as high-volume DDoS traffic. The motivation is that Random Forest can learn strong threshold-based decision rules over tabular features, while multilayer perceptrons can learn continuous feature interactions but may be more sensitive to class imbalance and dataset artifacts.

The theoretical contribution of this work is an attack-type-specific evaluation framework for interpreting flow-level IDS model behavior. Instead of treating model performance as one number, the study compares security failure modes. The empirical contribution is a reproducible implementation that trains six models on the same CSE-CIC-IDS2018 sample and reports both model-level and attack-level false negatives. The final result shows that neural models improve some aggregate metrics, but they do not consistently reduce missed attacks.

2 Related Work

Prior work on intrusion detection datasets motivates the empirical setting of this project. CICIDS2017 was introduced to provide a more realistic benchmark than older IDS datasets. It includes benign traffic and common attack traffic represented through labeled network-flow features [1]. CSE-CIC-IDS2018 extends this direction with a larger cyber-defense dataset containing multiple attack scenarios, including brute force, Heartbleed, botnet, DoS, DDoS, web attacks, and infiltration [2]. These datasets are directly relevant because they support both binary benign/malicious detection and attack-category-specific analysis. Prior work on IDS modeling motivates my choice of baselines. Random Forest is a standard traditional machine learning baseline because it combines many decision-tree predictors into an ensemble [3]. Deep learning has also been studied for network intrusion detection. Gamage and Samarabandu survey deep learning approaches for IDS and compare feed-forward neural networks, autoencoders, deep belief networks, and LSTMs on legacy and modern IDS datasets [4]. This supports the use of a lightweight multilayer perceptron while also showing that model comparison alone is not new. The gap this project addresses is

in how IDS model performance is interpreted. Many IDS studies emphasize overall accuracy or aggregate detection performance [4], but aggregate metrics can hide security-relevant failures when datasets are imbalanced or when some attack categories are harder to detect. Saito and Rehmsmeier show that precision-recall analysis is especially informative for imbalanced classification problems [5]. This matters for IDS because a false negative represents malicious traffic that the system fails to flag. Therefore, this project fits within existing work but asks a narrower and more security-centered question. I am not claiming that PyTorch will simply “do IDS better” than traditional machine learning. Instead, I will compare Logistic Regression, Random Forest, and a lightweight PyTorch MLP to identify which attack categories each model misses. The related work supports the project in three ways: CICIDS datasets provide the labeled flow-level setting, prior ML/DL IDS work justifies the model comparison, and imbalanced-evaluation research motivates recall and false-negative rate as the main security metrics.

3 Method

3.1 Dataset and Filtering

The study uses the CSE-CIC-IDS2018 processed traffic data for machine learning. The raw files were downloaded from the public AWS S3 bucket and processed into a reproducible sampled dataset. The final sample contains 50,000 flows, 79 numeric flow features, 37,308 benign flows, and 12,692 attack flows. The preprocessing step preserves 14 fine-grained attack labels so that binary model outputs can later be analyzed by attack type.

The task used for model training is binary classification. Each row is labeled as benign or malicious. The original attack label is not used as the prediction target, because the research question is about whether binary IDS models miss different types of attacks. Instead, the original label is carried through the train, validation, and test split and used only for post-hoc evaluation.

The preprocessing pipeline cleans column names, detects the label column, removes non-feature metadata columns, replaces missing and infinite values, scales numeric features, and splits the data into training, validation, and test sets. The final split contains 32,000 training rows, 8,000 validation rows, and 10,000 test rows. The random seed is fixed at 42 for reproducibility. Rare-label preservation is used during sampling so that small attack categories are not accidentally removed.

3.2 System Building Blocks

The implemented system is a lightweight flow-level IDS prototype. It has four main building blocks. First, the dataset preparation script downloads or reads raw CSE-CIC-IDS2018 CSV files, samples rows while preserving attack labels, and writes a processed training CSV. Second, the training pipeline loads the processed CSV, applies the shared preprocessing pipeline, trains all selected models, and saves model artifacts. Third, the evaluation module generates binary metrics, confusion matrices, attack-type detection summaries, attack-family detection summaries, and plots. Fourth, the prediction module loads saved artifacts and applies a trained model to a new CSV file.

The system is implemented in Python using pandas, NumPy, scikit-learn, PyTorch, and matplotlib. scikit-learn is used for Logistic Regression, Random Forest, scaling, splitting, and metric computation. PyTorch is used for the neural models and training loops. The repository also includes a README with commands for reproducing data preparation, training, and prediction.

3.3 Models

Six models are compared. Logistic Regression is the simple linear baseline and uses balanced class weights. Random Forest is the traditional non-neural baseline and uses balanced subsampling. The neural baselines are Shallow MLP, Lightweight PyTorch MLP, Deep MLP, and Autoencoder + MLP. The Shallow MLP has one hidden layer. The Lightweight PyTorch MLP is the original two-hidden-layer model. The Deep MLP uses more feed-forward layers to test whether depth improves attack recall. The Autoencoder + MLP first learns a compressed latent representation and then feeds that representation to a classifier.

All models are trained and evaluated on the same split. This keeps the comparison focused on model behavior rather than data differences. The neural models are not heavily tuned, because the goal is a scoped comparison of attack-specific failure patterns rather than a leaderboard-style optimization.

3.4 Measures

The main metrics are precision, recall, F1-score, false-negative rate, confusion matrix counts, attack-type detection rate, and attack-family detection rate. For the binary malicious class, recall measures the fraction of attacks detected. False-negative rate is one minus recall and directly measures the fraction of attacks missed. Precision measures how many flows predicted as malicious were actually malicious. F1-score balances precision and recall.

Attack-type detection rate is computed by grouping malicious test rows by their original fine-grained attack label and measuring the fraction of rows detected as malicious by each model. Attack-family detection rate maps fine-grained labels into five broader groups: Botnet, Brute Force, DoS/DDoS, Infiltration, and Web Attack. These two levels of analysis are both necessary. The fine-grained view shows specific failure cases, while the family-level view summarizes broader security patterns.

3.5 Implementation and Reproducibility

The experiment was executed as a reproducible command-line pipeline rather than as a manually edited notebook. This matters because the project compares several models and many attack categories; manual steps would make it easy to accidentally evaluate different models on different splits. The repository contains separate scripts for dataset preparation, training, evaluation, and prediction. The dataset preparation command reads the raw CSE-CIC-IDS2018 CSV files, applies rare-label-preserving sampling, and writes a processed CSV. The training command then reads that processed file, creates the fixed train/validation/test split, trains all models, saves artifacts, and writes result tables and plots.

The final execution used the CSE-CIC-IDS2018 processed machine-learning CSV files downloaded from the public AWS bucket. The raw local dataset contained 10 CSV files totaling about 6.41 GB. The

Table 1: Main implementation outputs from the final run.

Output	Purpose
metrics_summary.csv	Overall model metrics
attack_type_detection.csv	Fine-grained attack recall
attack_family_detection.csv	Coarse family recall
confusion_matrix_*.png	Model error visualization
false_negative_rate.png	Security-focused comparison
*.pt, *.pkl artifacts	Saved trained models

Table 2: Overall malicious-class performance on the test split.

Model	Precision	Recall	F1	FNR
Logistic Regression	0.512	0.528	0.520	0.472
Random Forest	0.649	0.471	0.546	0.529
Shallow MLP	0.578	0.524	0.550	0.476
PyTorch MLP	0.615	0.507	0.556	0.493
Deep MLP	0.747	0.467	0.574	0.533
Autoencoder + MLP	0.723	0.471	0.570	0.529

processed experiment sample was saved as `data/processed/ids_sample.csv`. The run produced model artifacts for Logistic Regression, Random Forest, Shallow MLP, PyTorch MLP, Deep MLP, and Autoencoder + MLP. It also produced `metrics_summary.csv`, `attack_type_detection.csv`, `attack_family_detection.csv`, confusion matrices, false-negative plots, and prediction outputs.

Table 1 summarizes the main execution outputs. These outputs provide evidence that the project was executed according to the planned method: the same dataset split was used across models, attack labels were preserved for evaluation, and both aggregate and attack-specific results were collected. The prediction script can also load a saved model and apply it to a CSV file, which demonstrates the system-building component beyond offline reporting.

4 Results

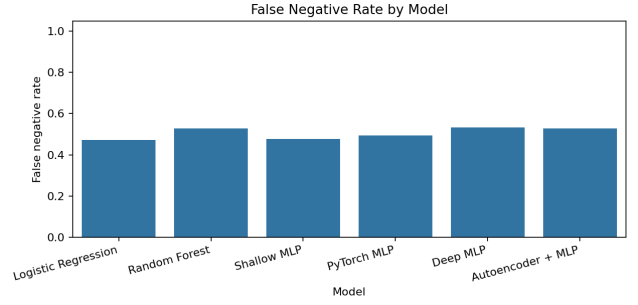
4.1 Overall Model Performance

Table 2 reports binary malicious-class performance on the 10,000-row test split. Deep MLP achieves the highest F1-score and precision. However, Logistic Regression achieves the highest recall and the lowest false-negative rate. This distinction is central to the security interpretation of the experiment. The model that looks best by F1-score is not the model that misses the fewest attacks.

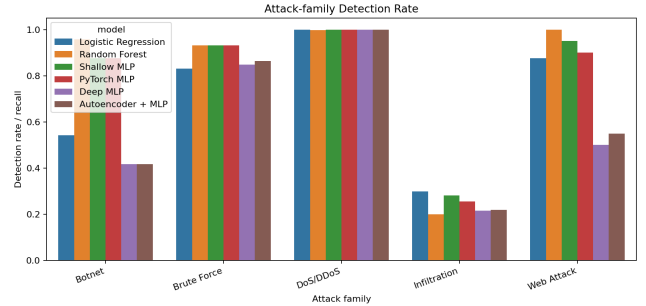
The false-negative counts make this clearer. Logistic Regression misses 1,197 attacks. Shallow MLP misses 1,209, PyTorch MLP misses 1,252, Random Forest misses 1,342, Autoencoder + MLP misses 1,343, and Deep MLP misses 1,354. Deep MLP is therefore more selective: when it predicts malicious, it is more often correct, but it leaves more attacks undetected. For an IDS, that tradeoff can be unacceptable if the operational priority is to minimize missed attacks.

4.2 Coarse Attack-Family Results

Table 3 summarizes the strongest model for each coarse attack family. The family-level results show that the difficulty of the IDS task depends strongly on the attack type. DoS/DDoS is nearly solved

**Figure 1: False-negative rate by model. Lower is better for IDS security coverage.****Table 3: Best model by coarse attack-family recall.**

Family	Best model	Recall
Botnet	Random Forest	0.958
Brute Force	RF / Shallow MLP / PyTorch MLP	0.932
DoS/DDoS	Shallow MLP / PyTorch MLP	1.000
Infiltration	Logistic Regression	0.299
Web Attack	Random Forest	1.000

**Figure 2: Detection recall by coarse attack family and model.**

by the tested models. Shallow MLP and PyTorch MLP both reach 1.000 recall on the DoS/DDoS family. Random Forest is strongest on Botnet and Web Attack. For Brute Force, Random Forest, Shallow MLP, and PyTorch MLP tie at 0.932 recall.

The most important result is the Infiltration family. It has 1,669 test attacks, but even the best model, Logistic Regression, detects only 499 of them. The family-level recall is only 0.299. Random Forest detects 334, Shallow MLP detects 469, PyTorch MLP detects 428, Deep MLP detects 359, and Autoencoder + MLP detects 367. This family drives a large portion of the overall false negatives.

4.3 Fine-Grained Attack-Type Results

The fine-grained results explain which specific attacks create the family-level patterns. Several attack labels are detected almost perfectly by nearly all models. DDOS attack-HOIC, DDOS attack-LOIC-UDP, DoS attacks-SlowHTTPTest, FTP-BruteForce, and SSH-BruteForce all have near-perfect or perfect recall across most models.

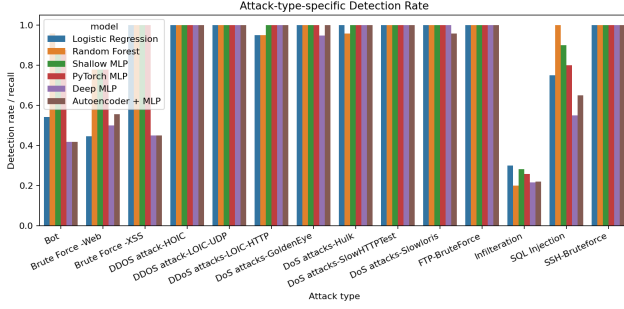


Figure 3: Fine-grained attack-type recall by model.

These attacks appear to have flow-feature signatures that are easier to separate from benign traffic.

Table 4 reports the per-label recall values used for this analysis. The table is useful because the best model changes by attack type. It also shows that the headline difficulty is not evenly distributed across the dataset. Some rows are almost entirely solved, while Infiltration remains consistently weak across all six models.

DDoS attacks-LOIC-HTTP is one case where the neural models are slightly more stable. Shallow MLP, PyTorch MLP, Deep MLP, and Autoencoder + MLP all achieve 1.000 recall, while Logistic Regression and Random Forest each miss one sample. This supports the hypothesis in a limited sense: there are some attack types where neural models match or slightly improve the traditional baselines.

Bot traffic shows the opposite pattern. Random Forest reaches 0.958 recall, while Logistic Regression reaches 0.542 and the deeper neural models perform poorly. Deep MLP and Autoencoder + MLP each reach only 0.417 recall on Bot. This suggests that a tree-based model may capture useful feature thresholds for this small category better than the deeper neural models used in this experiment.

Brute Force -Web also favors models that can represent feature interactions. Random Forest, Shallow MLP, and PyTorch MLP all reach 0.778 recall, while Logistic Regression reaches 0.444. However, Brute Force -XSS shows that deeper neural models are not automatically more robust on rare labels. Logistic Regression, Random Forest, Shallow MLP, and PyTorch MLP reach 1.000 recall, but Deep MLP and Autoencoder + MLP reach only 0.450.

SQL Injection is strongest under Random Forest, which reaches 1.000 recall. Shallow MLP is also strong at 0.900, while Logistic Regression reaches 0.750, PyTorch MLP reaches 0.800, Deep MLP reaches 0.550, and Autoencoder + MLP reaches 0.650. This supports the broader result that Random Forest remains competitive and sometimes superior for web-related attacks.

The most important fine-grained attack label is Infiltration. It is the largest attack label in the test set, with 1,669 samples, and every model performs poorly. Logistic Regression is best at 0.299 recall. Shallow MLP reaches 0.281, PyTorch MLP reaches 0.256, Autoencoder + MLP reaches 0.220, Deep MLP reaches 0.215, and Random Forest reaches 0.200. This is the clearest security blind spot in the experiment.

5 Discussion

The results partially support the hypothesis. Neural models improve some aggregate measures, and Shallow MLP and PyTorch MLP match Random Forest on the Brute Force family. The MLP models also perform very well on DoS/DDoS traffic. However, the neural models do not reduce false negatives overall. The highest-recall model is Logistic Regression, and the lowest-recall models include the deeper neural variants.

This result matters because the metric choice changes the conclusion. If the project reported only F1-score, Deep MLP would look like the strongest model. If the project reported only precision, Deep MLP would look even better. But the security objective of an IDS is not only to make accurate malicious predictions. It is also to avoid allowing malicious traffic to pass undetected. By that standard, Logistic Regression and Shallow MLP are more attractive than Deep MLP in this experiment.

The attack-type breakdown also shows why aggregate metrics can be misleading. DoS/DDoS categories are detected almost perfectly, and because they have clearer patterns, they can make the overall system appear strong. At the same time, Infiltration is both frequent and difficult. Every model misses most Infiltration samples. Therefore, a deployment decision based only on aggregate F1 could hide the exact category where the IDS needs the most improvement.

The Autoencoder + MLP result is also informative. Autoencoders are common in deep IDS literature because they can learn compact representations of high-dimensional flow features. In this experiment, the autoencoder-based model behaves similarly to Deep MLP: high precision, relatively high F1, but low recall. This does not prove that autoencoders are ineffective for IDS. It shows that this scoped representation-learning baseline did not solve the false-negative problem.

6 Risks and Limitations

The first limitation is dataset dependence. CSE-CIC-IDS2018 is a public benchmark dataset, and models may learn dataset-specific artifacts rather than general intrusion behavior. To reduce this risk, the study avoids accuracy-only reporting and focuses on false negatives by attack type. However, the results should still be interpreted as behavior on this sampled benchmark, not as a production IDS guarantee.

The second limitation is class imbalance. Some attack types have small test support, so perfect recall on those labels should not be overclaimed. For example, several rare attack categories have fewer than 30 test samples. The analysis therefore gives special attention to Infiltration because it is both frequent and poorly detected.

The third limitation is model tuning. The neural models were implemented as reproducible baselines rather than exhaustively tuned architectures. Different thresholds, loss functions, resampling strategies, or hyperparameters might improve recall. This is especially important because the default 0.5 decision threshold may not be optimal for security.

The fourth limitation is the binary training objective. The models are trained to distinguish benign from malicious flows, and attack labels are used only after training for analysis. A multiclass or hierarchical model might learn attack-specific patterns more directly.

Table 4: Fine-grained attack-type recall by model. LR = Logistic Regression, RF = Random Forest, Shallow = Shallow MLP, PT = PyTorch MLP, Deep = Deep MLP, AE = Autoencoder + MLP.

Attack type	Test	LR	RF	Shallow	PT	Deep	AE
Bot	24	0.542	0.958	0.875	0.875	0.417	0.417
Brute Force -Web	18	0.444	0.778	0.778	0.778	0.500	0.556
Brute Force -XSS	20	1.000	1.000	1.000	1.000	0.450	0.450
DDoS attack-HOIC	618	1.000	1.000	1.000	1.000	1.000	1.000
DDoS attack-LOIC-UDP	15	1.000	1.000	1.000	1.000	1.000	1.000
DDoS attacks-LOIC-HTTP	20	0.950	0.950	1.000	1.000	1.000	1.000
DoS attacks-GoldenEye	19	1.000	1.000	1.000	1.000	0.947	1.000
DoS attacks-Hulk	23	1.000	0.957	1.000	1.000	1.000	1.000
DoS attacks-SlowHTTPTest	28	1.000	1.000	1.000	1.000	1.000	1.000
DoS attacks-Slowloris	23	1.000	1.000	1.000	1.000	1.000	0.957
FTP-BruteForce	18	1.000	1.000	1.000	1.000	1.000	1.000
Infiltration	1669	0.299	0.200	0.281	0.256	0.215	0.220
SQL Injection	20	0.750	1.000	0.900	0.800	0.550	0.650
SSH-Bruteforce	23	1.000	1.000	1.000	1.000	1.000	1.000

The binary design was chosen because it matches a lightweight IDS prototype and keeps the comparison focused.

7 Future Work

The first next step is threshold tuning for security objectives. Instead of using a fixed 0.5 threshold, each model should be evaluated under thresholds chosen to reduce false negatives while controlling false positives. This would directly test whether Deep MLP and Autoencoder + MLP can trade some precision for better recall.

The second next step is targeted improvement for Infiltration. Because Infiltration dominates the missed attacks, future work should inspect its feature distribution, compare it against benign flows, and test whether resampling or cost-sensitive training improves detection. This is a higher priority than adding more model types because it addresses the main security blind spot.

The third next step is multiclass or hierarchical classification. A model could first detect malicious traffic and then classify the attack family. Alternatively, attack-family labels could be used during training to encourage the model to learn category-specific features.

The fourth next step is broader validation. The current study uses CSE-CIC-IDS2018. Running the same pipeline on CICIDS2017 or another flow-level IDS dataset would show whether the attack-specific conclusions generalize beyond one benchmark.

8 Conclusion

This paper evaluated whether neural models reduce missed attacks in flow-level intrusion detection compared with traditional machine learning baselines. The answer is mixed. Deep MLP and Autoencoder + MLP improve precision and F1-score, but Logistic Regression has the highest attack recall and lowest false-negative rate. Shallow MLP is the most stable neural model by recall and performs close to Logistic Regression.

The strongest conclusion is that IDS evaluation must be attack-type-specific. DoS/DDoS and several brute-force attacks are detected well, while Infiltration is difficult for every model. Therefore, aggregate metrics alone are not enough. A model that looks strong by F1-score may still miss the attack category that creates the greatest security risk.

References

- [1] [1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani. 2018. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy*, 108–116. <https://www.unb.ca/cic/datasets/ids-2017.html>
- [2] [2] Canadian Institute for Cybersecurity. 2018. CSE-CIC-IDS2018 on AWS: A Realistic Cyber Defense Dataset. University of New Brunswick. <https://www.unb.ca/cic/datasets/ids-2018.html>
- [3] [3] L. Breiman. 2001. Random Forests. *Machine Learning* 45, 1, 5–32. <https://doi.org/10.1023/A:1010933404324>
- [4] [4] S. Gamage and J. Samarabandu. 2020. Deep Learning Methods in Network Intrusion Detection: A Survey and an Objective Comparison. *Journal of Network and Computer Applications* 169, 102767.
- [5] [5] T. Saito and M. Rehmsmeier. 2015. The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLOS ONE* 10, 3, e0118432. <https://doi.org/10.1371/journal.pone.0118432>